StockFlow - Backend Engineering Intern

Case Study Response

Submitted by: Satyam Gaikwad
Date: 5th May 2026
Position: Backend Engineering Intern

Part 1: Code Review & Debugging

1.1 Issues Identified

Issue 1: No Input Validation

The code directly accesses data['name'], data['sku'], etc. without checking if the keys exist or if the values are valid. If any field is missing, the endpoint will throw a KeyError (500 Internal Server Error). There is no type checking either — price could be a string, quantity could be negative.

Production Impact:

Any malformed request crashes the server with an unhandled exception. Clients receive unhelpful 500 errors instead of meaningful validation messages. Negative prices or quantities could corrupt business data.

Issue 2: No Duplicate SKU Check

SKUs must be unique across the platform (per the requirements), but the code never checks if a product with the same SKU already exists. The database might have a unique constraint that would throw an IntegrityError, but this is unhandled.

Production Impact:

If the DB has a unique constraint, the entire transaction fails with an unhandled exception. If there is no constraint, duplicate SKUs are created, breaking inventory lookups and reports.

Issue 3: No Transaction Atomicity

The code calls db.session.commit() twice — once after creating the product and once after creating the inventory record. If the second commit fails (e.g., database error, constraint violation), the product exists in the database but has no inventory record. This leaves the system in an inconsistent state.

Production Impact:

Orphaned products without inventory records. The warehouse shows a product but with no stock information. Requires manual database cleanup.

Issue 4: No Error Handling / Try-Except

There is zero exception handling. Any database error, connection issue, or constraint violation results in an unhandled 500 error with a potential stack trace leak.

Production Impact:

Stack traces could leak sensitive database schema information to clients. No graceful degradation. Hard to debug in production without proper error logging.

Issue 5: Missing HTTP Status Codes

The endpoint returns a JSON response but does not set an appropriate HTTP status code. A successful creation should return 201 Created, not the default 200 OK. Error cases don't return proper 4xx/5xx codes.

Production Impact:

API consumers (frontend apps, integrations) cannot reliably determine the outcome of their request. Breaks RESTful API conventions.

Issue 6: No Authentication / Authorization

There is no check on who is making the request. Any unauthenticated user could create products in any warehouse. There is no warehouse ownership validation either — a user from Company A could add products to Company B's warehouse.

Production Impact:

Critical security vulnerability. Data can be tampered with by unauthorized users. Complete data integrity compromise in a multi-tenant B2B system.

Issue 7: Price Precision (Floating Point)

If price is stored as a float, decimal precision issues will occur (e.g., 19.99 might become 19.989999...). Financial values should use Decimal/Numeric types.

Production Impact:

Rounding errors in pricing, invoicing, and revenue calculations. Small errors accumulate over thousands of transactions.

1.2 Corrected Code

```
from flask import request, jsonify

from decimal import Decimal, InvalidOperation

from sqlalchemy.exc import IntegrityError

import logging


logger = logging.getLogger(__name__)


@app.route('/api/products', methods=['POST'])

@login_required  # Authentication decorator

def create_product():

    data = request.json


    # --- Input Validation ---

    required_fields = ['name', 'sku', 'price', 'warehouse_id',
'initial_quantity']

    missing = [f for f in required_fields if f not in data or data[f] is
None]

    if missing:

        return jsonify({

            "error": "Missing required fields",

            "missing_fields": missing

        }), 400


    # Validate data types and business rules
```

```
    try:
        price = Decimal(str(data['price']))
        if price <= 0:
            return jsonify({"error": "Price must be positive"}), 400
    except (InvalidOperation, ValueError):
        return jsonify({"error": "Invalid price format"}), 400

    quantity = data['initial_quantity']
    if not isinstance(quantity, int) or quantity < 0:
        return jsonify({"error": "Quantity must be a non-negative integer"}),
400

    # --- Check warehouse exists and belongs to user's company ---
    warehouse = Warehouse.query.get(data['warehouse_id'])
    if not warehouse or warehouse.company_id != current_user.company_id:
        return jsonify({"error": "Invalid warehouse"}), 404

    # --- Check duplicate SKU ---
    existing = Product.query.filter_by(sku=data['sku']).first()
    if existing:
        return jsonify({"error": "SKU already exists"}), 409

    # --- Atomic Transaction ---
    try:
        product = Product(
            name=data['name'].strip(),
```

```
            sku=data['sku'].strip().upper(),
            price=price,
            warehouse_id=data['warehouse_id']
        )
        db.session.add(product)
        db.session.flush()  # Get product.id without committing

        inventory = Inventory(
            product_id=product.id,
            warehouse_id=data['warehouse_id'],
            quantity=quantity
        )
        db.session.add(inventory)
        db.session.commit()  # Single commit for both operations

        logger.info(f"Product created: {product.id}, SKU: {product.sku}")

        return jsonify({
            "message": "Product created successfully",
            "product_id": product.id,
            "sku": product.sku
        }), 201

    except IntegrityError:
        db.session.rollback()
```

```
        logger.error(f"IntegrityError creating product SKU: {data['sku']}")

        return jsonify({"error": "Product could not be created (duplicate
entry)"}), 409



    except Exception as e:

        db.session.rollback()

        logger.exception("Unexpected error creating product")

        return jsonify({"error": "Internal server error"}), 500
```

1.3 Key Fixes Summary

Issue	Fix Applied	Benefit
No validation	Comprehensive input checks	Prevents bad data entry
No SKU uniqueness	Pre-check + IntegrityError handling	No duplicate products
Two commits	flush() + single commit()	Atomic operation
No error handling	try/except with rollback	Graceful failure + logging
Wrong status code	201 for success, 4xx for errors	RESTful compliance
No auth	@login_required + ownership check	Multi-tenant security
Float price	Decimal type	Accurate financials

Part 2: Database Design

2.1 Entity-Relationship Overview

The schema is designed around the core entities: Companies, Warehouses, Products, Suppliers, Inventory, and Inventory Change Logs. It supports multi-tenancy (each company sees only their data), product bundles, and full audit trails for inventory changes.

2.2 SQL DDL Schema

```
-- 1. Companies Table

CREATE TABLE companies (

    id              SERIAL PRIMARY KEY,

    name            VARCHAR(255) NOT NULL,

    email           VARCHAR(255) UNIQUE NOT NULL,

    phone           VARCHAR(20),

    address         TEXT,

    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);



-- 2. Warehouses Table

CREATE TABLE warehouses (

    id              SERIAL PRIMARY KEY,

    company_id      INT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,

    name            VARCHAR(255) NOT NULL,

    location        TEXT,

    is_active       BOOLEAN DEFAULT TRUE,

    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);
```

```
CREATE INDEX idx_warehouses_company ON warehouses(company_id);


-- 3. Products Table
CREATE TABLE products (
    id              SERIAL PRIMARY KEY,
    company_id      INT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
    name            VARCHAR(255) NOT NULL,
    sku             VARCHAR(100) UNIQUE NOT NULL,
    description     TEXT,
    price           NUMERIC(12, 2) NOT NULL CHECK (price >= 0),
    product_type    VARCHAR(50) DEFAULT 'standard',  -- 'standard', 'bundle',
'perishable'
    low_stock_threshold INT DEFAULT 10,
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_products_company ON products(company_id);
CREATE INDEX idx_products_sku ON products(sku);


-- 4. Product Bundles (self-referencing many-to-many)
CREATE TABLE product_bundles (
    id              SERIAL PRIMARY KEY,
    bundle_id       INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    component_id    INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    quantity         INT NOT NULL DEFAULT 1 CHECK (quantity > 0),
```

```
    UNIQUE(bundle_id, component_id),
    CHECK (bundle_id != component_id)  -- Prevent self-reference
);

-- 5. Suppliers Table
CREATE TABLE suppliers (
    id              SERIAL PRIMARY KEY,
    name            VARCHAR(255) NOT NULL,
    contact_email   VARCHAR(255),
    contact_phone   VARCHAR(20),
    address         TEXT,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 6. Company-Supplier Relationship (many-to-many)
CREATE TABLE company_suppliers (
    id              SERIAL PRIMARY KEY,
    company_id      INT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
    supplier_id     INT NOT NULL REFERENCES suppliers(id) ON DELETE CASCADE,
    UNIQUE(company_id, supplier_id)
);

-- 7. Product-Supplier Mapping (which supplier provides which product)
CREATE TABLE product_suppliers (
    id              SERIAL PRIMARY KEY,
    product_id      INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
```

```
    supplier_id     INT NOT NULL REFERENCES suppliers(id) ON DELETE CASCADE,
    unit_cost       NUMERIC(12, 2),
    lead_time_days  INT,  -- Delivery lead time
    is_preferred    BOOLEAN DEFAULT FALSE,
    UNIQUE(product_id, supplier_id)
);

-- 8. Inventory Table (product quantities per warehouse)
CREATE TABLE inventory (
    id              SERIAL PRIMARY KEY,
    product_id      INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    warehouse_id    INT NOT NULL REFERENCES warehouses(id) ON DELETE CASCADE,
    quantity         INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    last_updated    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(product_id, warehouse_id)
);
CREATE INDEX idx_inventory_product ON inventory(product_id);
CREATE INDEX idx_inventory_warehouse ON inventory(warehouse_id);

-- 9. Inventory Change Log (audit trail)
CREATE TABLE inventory_logs (
    id              SERIAL PRIMARY KEY,
    inventory_id    INT NOT NULL REFERENCES inventory(id),
    product_id      INT NOT NULL REFERENCES products(id),
    warehouse_id    INT NOT NULL REFERENCES warehouses(id),
```

```
    change_type     VARCHAR(50) NOT NULL,  -- 'sale', 'restock', 'transfer',
'adjustment'

    quantity_change  INT NOT NULL,  -- Positive for additions, negative for
removals

    quantity_after   INT NOT NULL,

    reference_id    VARCHAR(100),  -- Order ID, PO number, etc.

    notes           TEXT,

    created_by      INT,  -- User who made the change

    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

CREATE INDEX idx_inv_logs_product ON inventory_logs(product_id);

CREATE INDEX idx_inv_logs_created ON inventory_logs(created_at);
```

2.3 Design Decisions & Justifications

1. NUMERIC(12,2) for price: Avoids floating-point precision errors. Supports values up to 9,999,999,999.99 which is sufficient for any inventory item.
2. Separate inventory table with UNIQUE(product_id, warehouse_id): Allows the same product to exist in multiple warehouses with independent quantities, as required.
3. inventory_logs as an append-only audit trail: Every stock change is recorded with who, when, why, and the resulting quantity. Essential for compliance and debugging discrepancies.
4. product_bundles with self-referencing foreign keys: A bundle is itself a product, so we use a junction table linking a bundle product to its component products. The CHECK constraint prevents a product from being a component of itself.
5. product_suppliers junction table: Allows multiple suppliers per product with different costs and lead times. The is_preferred flag enables quick lookup of the primary supplier for reordering.
6. Indexes on foreign keys and frequently filtered columns: company_id, sku, product_id on inventory, and created_at on logs are indexed for fast queries. The low-stock alert query specifically benefits from these.
7. Soft deletes via is_active flags: Products and warehouses aren't hard-deleted because historical inventory logs reference them. Soft deletes preserve referential integrity.

2.4 Questions for the Product Team (Identified Gaps)

1. Should bundles auto-deduct component inventory when sold, or are they tracked independently?
2. Do we need to support inter-warehouse transfers? (I've included 'transfer' as a change_type assuming yes.)
3. Is there a concept of 'reserved' or 'committed' stock (e.g., items in pending orders)?

4. Should low_stock_threshold be per-product or per-product-per-warehouse?
5. Do suppliers serve the entire platform or are they company-specific?
6. Are there different user roles (admin, warehouse manager, viewer) that need RBAC?
7. Do we need to support multiple currencies for international suppliers?
8. Is there a maximum number of warehouses per company? Any plan/tier limits?

Part 3: API Implementation — Low-Stock Alerts

3.1 Assumptions

- The database schema from Part 2 is used.
- "Recent sales activity" means at least one sale-type inventory_log entry in the last 30 days.
- days_until_stockout is calculated using average daily sales rate over the last 30 days.
- The preferred supplier (is_preferred = True) is returned; if none is preferred, the first available supplier is used.
- Pagination is supported via query parameters (page, per_page).
- Authentication is handled by a decorator that provides current_user.

3.2 Implementation (Python / Flask + SQLAlchemy)

```
from flask import request, jsonify

from datetime import datetime, timedelta

from sqlalchemy import func, and_

import math



@app.route('/api/companies/<int:company_id>/alerts/low-stock',
methods=['GET'])

@login_required

def get_low_stock_alerts(company_id):

    """

    Returns low-stock alerts for all warehouses belonging to a company.

    Only includes products with recent sales activity (last 30 days).

    """
```

```
# --- Authorization: Ensure user belongs to this company ---
if current_user.company_id != company_id:
    return jsonify({"error": "Unauthorized"}), 403

# --- Validate company exists ---
company = Company.query.get(company_id)
if not company:
    return jsonify({"error": "Company not found"}), 404

# --- Pagination parameters ---
page = request.args.get('page', 1, type=int)
per_page = request.args.get('per_page', 20, type=int)
per_page = min(per_page, 100)  # Cap at 100

# --- Define time window for 'recent activity' ---
thirty_days_ago = datetime.utcnow() - timedelta(days=30)

# --- Step 1: Find products with recent sales activity ---
# Subquery: product_ids that had sales in last 30 days
recent_sales_subquery = (
    db.session.query(InventoryLog.product_id)
    .filter(
        InventoryLog.change_type == 'sale',
        InventoryLog.created_at >= thirty_days_ago
```

```
    )
    .distinct()
    .subquery()
)

# --- Step 2: Query inventory below threshold with recent sales ---
low_stock_query = (
    db.session.query(
        Inventory, Product, Warehouse
    )
    .join(Product, Inventory.product_id == Product.id)
    .join(Warehouse, Inventory.warehouse_id == Warehouse.id)
    .filter(
        Warehouse.company_id == company_id,
        Warehouse.is_active == True,
        Product.is_active == True,
        Inventory.quantity <= Product.low_stock_threshold,
        Product.id.in_(recent_sales_subquery)
    )
    .order_by(Inventory.quantity.asc())  # Most urgent first
)

# --- Get total count before pagination ---
total_alerts = low_stock_query.count()

# --- Apply pagination ---
```

```
results = low_stock_query.offset(
    (page - 1) * per_page
).limit(per_page).all()

# --- Step 3: Build response with supplier info and stockout estimate ---
alerts = []
for inventory, product, warehouse in results:

    # Calculate average daily sales rate (last 30 days)
    total_sold = (
        db.session.query(
            func.coalesce(
                func.sum(func.abs(InventoryLog.quantity_change)), 0
            )
        )
        .filter(
            InventoryLog.product_id == product.id,
            InventoryLog.warehouse_id == warehouse.id,
            InventoryLog.change_type == 'sale',
            InventoryLog.created_at >= thirty_days_ago
        )
        .scalar()
    )

    daily_sales_rate = total_sold / 30.0 if total_sold > 0 else 0
```

```
# Estimate days until stockout
if daily_sales_rate > 0:
    days_until_stockout = math.floor(
        inventory.quantity / daily_sales_rate
    )
else:
    days_until_stockout = None  # No sales, can't estimate

# Get preferred supplier for this product
supplier_link = (
    ProductSupplier.query
    .filter_by(product_id=product.id)
    .order_by(ProductSupplier.is_preferred.desc())
    .first()
)

supplier_info = None
if supplier_link:
    supplier = Supplier.query.get(supplier_link.supplier_id)
    if supplier:
        supplier_info = {
            "id": supplier.id,
            "name": supplier.name,
            "contact_email": supplier.contact_email,
            "lead_time_days": supplier_link.lead_time_days
```

```
                }


        alerts.append({
            "product_id": product.id,
            "product_name": product.name,
            "sku": product.sku,
            "warehouse_id": warehouse.id,
            "warehouse_name": warehouse.name,
            "current_stock": inventory.quantity,
            "threshold": product.low_stock_threshold,
            "days_until_stockout": days_until_stockout,
            "daily_sales_rate": round(daily_sales_rate, 2),
            "supplier": supplier_info
        })


    return jsonify({
        "alerts": alerts,
        "total_alerts": total_alerts,
        "page": page,
        "per_page": per_page,
        "total_pages": math.ceil(total_alerts / per_page) if total_alerts > 0
else 0
    }), 200
```

3.3 Edge Cases Handled

Edge Case	How It's Handled
Company doesn't exist	Returns 404 with clear error message
User from different company	Returns 403 Unauthorized (multi-tenant security)
No warehouses / no products	Returns empty alerts array with total_alerts: 0
Product has no sales in 30 days	Excluded from alerts (filtered by recent_sales_subquery)
Zero daily sales rate	days_until_stockout set to null (cannot estimate)
No supplier linked to product	supplier field is null in response
Very large result sets	Pagination with configurable per_page, capped at 100
Inactive products/warehouses	Excluded via is_active = True filter

3.4 Performance Considerations

- The recent_sales_subquery runs once and is reused, avoiding N+1 queries for the sales check.
- Results are sorted by quantity ascending so the most critical alerts appear first.
- Pagination prevents loading thousands of alerts at once.
- Future optimization: Pre-compute daily_sales_rate via a scheduled job and cache it, avoiding repeated aggregation queries on each API call.
- Database indexes on inventory(product_id, warehouse_id) and inventory_logs(product_id, created_at) ensure efficient query execution.

Assumptions Made Due to Incomplete Requirements

1. A 'recent sales activity' window of 30 days is used to determine active products. This is configurable.
2. low_stock_threshold is a per-product setting stored in the products table. Could also be per-warehouse.
3. days_until_stockout is estimated using a simple average (total sold in 30 days / 30). A more sophisticated model could use weighted moving averages.
4. A product's preferred supplier is determined by the is_preferred flag in product_suppliers. If multiple are preferred, the first one returned by the database is used.
5. Authentication and authorization are handled by a decorator (@login_required) and current_user context, consistent with Flask-Login patterns.
6. The platform uses PostgreSQL as the database (DDL uses SERIAL, NUMERIC, BOOLEAN, etc.).
7. All timestamps are stored in UTC.

Alternative Approaches Considered

For Part 1 (Code Review)

- Could use marshmallow or pydantic for request validation instead of manual checks. Chose manual validation for clarity and fewer dependencies in an intern-level codebase.
- Could use database-level unique constraint + exception handling only (without pre-checking SKU). Chose pre-check for better user-facing error messages.

For Part 2 (Database Design)

- Considered a single products table with a parent_id for bundles (adjacency list). Chose a junction table (product_bundles) because it supports quantities per component and is more flexible.
- Considered an event-sourcing approach where inventory is derived entirely from logs. Chose a current-state table + change log for simpler queries and better read performance.

For Part 3 (API Implementation)

- Considered a single complex SQL query with JOINs for the entire alert (including daily sales rate). Chose a hybrid approach: SQL for filtering + Python for rate calculation, as it's more readable and debuggable.
- Considered using Redis caching for frequently accessed alert data. Deferred to keep the initial implementation simple, but noted it as a future optimization.